

Reverse Engineering DVFS Mechanisms

Ryan Piersma, Tawhid Bhuiyan, Tanvir Ahmed Khan, Simha Sethumadhavan

Columbia University, New York, NY, USA

ryan.piersma@cs.columbia.edu, {mb5332,tk3070,simha}@columbia.edu

Abstract—Dynamic Voltage and Frequency Scaling (DVFS) provides superlinear power savings to computer systems by lowering supply voltage and clock frequency when processors wait for other systems components such as memory and I/O, in addition to enforcing thermal chip constraints. More recent DVFS mechanisms are managed fully in hardware to reduce response times, but existing analysis specific to hardware-managed DVFS is lacking. In this study, we aim to reconstruct the DVFS mechanism implemented in modern hardware by constructing a custom high frequency sampling mechanism using a Linux kernel module that enables flexible and low overhead sampling of Model-Specific Registers (MSRs) derived from the power management mechanism. We study an extensive reverse engineering of Intel Core 3 100 U’s DVFS mechanism. Our study reveals two novel findings: (1) the mapping from abstract firmware standards to an Intel-specific implementation of fixed voltage-frequency states and (2) the function of undocumented bits in an MSR. We then demonstrate that we are able to perform a model stealing attack through a variety of machine-learned proxy models based on access to relevant hardware with 99% R^2 coefficient for regression models and up to 92% F1 score for a classifier.

Index Terms—Power management, DVFS, reverse engineering, machine learning models

I. INTRODUCTION

Power management is a significant product differentiator for general purpose computing due to increasing transistor counts coupled with a slowdown in Dennard scaling. In recent years, the efficiency of power management has increased. A key contributor to this increased efficiency is from improvements to power management mechanisms. Dynamic Voltage and Frequency Scaling (DVFS) aims to modulate the processor’s voltage and frequency to reduce power consumption without impacting application throughput or latency (for instance, on a memory miss when the processor is stalled, it can run at a lower voltage and frequency). Over the past few years, DVFS is managed more and more in hardware to reduce response times and increase potential savings.

Recently, processor vendors have alluded to the use of machine learning models in their power management mechanisms (as an example, Intel’s Meteor Lake processors [28]). In this work, we examine if these power management models can be reconstructed with sufficient fidelity by observing only the input-output behavior of power management schemes. An adversary may be motivated to reconstruct these models for economic reasons or to help conduct attacks. For an economically motivated adversary, reconstructed models offer a path to shortcut the costly process of research, data collection, cleaning, and training, and eventually reduce the time to market, to the detriment of the company that made the investments in the

first place. For adversaries who are interested in conducting attacks on power management systems, these reconstructed models offer a convenient way to understand model behavior and can enable faster identification of model blindspots that can be used for exploitation.

To steal a proprietary DVFS scheme, adversaries need to collect enough data at the right granularity to train the proxy models. We demonstrate this process by collecting this information on an Intel Core 3 100U processor [9]: we confirm the functionality of a Model-Specific Register (MSR) bitfield representing processor frequency, uncover Intel’s mapping from an abstract firmware standard to physical setpoints for voltage and frequency, and explore the parameters governing the processor’s responsiveness to workloads, which provides the necessary data sampling rate to fully reconstruct workload power traces.

Next, to be able to create high fidelity replica models, we recognize the need for very high sampling rates, which are not sufficiently provided by existing performance tools. Further, as dynamic power management relies on (roughly) system utilization, we need to simultaneously collect counters from relevant hardware performance events to be used as model inputs *without significantly perturbing the utilization*. To this end, we create a Linux kernel module that leverages precise kernel timing mechanisms and privileged instruction access to capture the necessary granularity for hardware-managed DVFS while reducing sampling overheads to acceptable rates.

We make use of our data to construct several machine learning models including linear regression, deep neural networks (DNNs), recurrent neural networks (RNNs), and gradient boosted decision trees, to track key microarchitectural events and output precise power measurements. We focus on a range of small models whose architectural complexity and size (lower bound $\approx 500B$) we deem acceptable for hardware embedding. Our results show that even under these tight constraints proxy models have significant reproducibility.

We make the following contributions in this paper:

- 1) A case study in reverse engineering the DVFS power management mechanism of a contemporary processor: an Intel Core 3 100U chip [9]
- 2) A Linux kernel module mechanism for sampling arbitrary sets of performance counters and MSRs with low performance overhead enabling direct data collection on real processor workloads
- 3) A demonstration across wide classes of machine learning models that an accurate data collection mechanism leaves machine-learning driven power controllers sus-

ceptible to model stealing attacks in only 400 training epochs, fitting regression models with an R^2 over 99% and classifiers with an F1 score of 92%, with average deviations within the gap between discrete voltage/frequency settings in the processor

We now briefly explain the structure of the paper. Section II provides necessary background for our work. In Section III, we describe the process of reverse engineering a modern Intel processor’s power management mechanism. In Section IV, we describe our data collection and associated methodology. In Section V, we display the machine learned models we built from the data followed by using the results to perform model stealing attacks in Section VI and briefly describe and briefly describe an additional application to power side channel attacks on cryptographic keys in Section VII. Finally, in Sections VIII and IX we discuss related work and conclude the paper.

II. BACKGROUND

A. Processor Power Consumption

In this study we primarily focus on the power consumption of a single processor core. The total power processor cores consume includes two components: (1) static power and (2) dynamic power.

$$P_{total} = P_s + P_d \quad (1)$$

Considering only a single core, the static component (P_s) represents the power consumed when the core is at rest. Static power is due to current that *leaks* through transistors even when there is no activity in the processor core. The core consumes this static power despite some or most of its associated components not performing any computation. The dynamic component represents when the core is fully engaged, with all of its components consuming power by performing computations. Compared to the constant static leakage, dynamic power consumed varies with several factors:

$$P_d = CV^2 f \alpha \quad (2)$$

C represents chip capacitance, V represents the voltage across the chip, f represents the primary clock frequency at which the chip is driven, and α represents the switching factor, a number between 0 and 1 corresponding to the relative frequency of transistors being switched on or off. Of these factors, V and f are the only directly configurable values. Thus, any attempt to control the power consumed by the chip during active operation should involve voltage and frequency.

B. DVFS

Weiser et al. proposed Dynamic Voltage and Frequency Scaling (DVFS) in 1994 for dynamic power management [34]. They observed that V should scale proportionally to f because higher frequencies require lower transistor gate settling times, which is best managed by a corresponding increase in voltage. This motivates the question of how tandem control of voltage and frequency corresponds to energy savings. DVFS captures this by increasing performance per watt of power. To illustrate,

simply reducing frequency alone would reduce the rate of computation as well as power linearly, thus not improving performance per watt. Reducing the voltage together with the frequency reduces the rate of work by δ but decreases power by δ^3 , thus enabling instantaneous performance per watt improvements proportional to δ^2 . The heuristic used to enable savings is that the processor can afford to have its voltage and frequency lowered when the processor needs to wait for slower devices like memory or I/O. However, as chip technology has continued to evolve, numerous changes in system level performance have motivated researchers to question whether DVFS is more effective than simply enabling ultra-efficient sleep states [35],[29] for the processor and always running the processor at maximal voltage and frequency when active.

C. Processor Power Management in Practice

1) *The Power Management Control Loop*: To implement the management of voltage and frequency in the processor, high speed circuit elements connected in a feedback loop with measurements of the thermal state of the processor, utilization of the processor, and activeness of other cores in a multicore system continually make adjustments, ideally in an energy efficient way. Voltage changes are effected by voltage regulators, which are now commonly integrated on-chip and typically at a per-core granularity. On-chip voltage regulators are typically low-dropout regulators (LDOs) that calibrate the voltage based on the current induced by the primary logic circuits of the processor using a variable resistance. Frequency changes are effected by Phase Lock Loops (PLLs), whose base frequency can be converted to many linear combinations of multiples in what is called a “fractional-N” configuration.

2) *Operating System-Driven Power Management*: Older generations of hardware used the operating system to direct DVFS mechanisms, with mechanisms such as the Linux kernel’s CPUFreq stack [11], which implements three layers: “core” software meant to provide utilities, “governor” software meant to provide methods of making power management decisions, and “driver” software that directly interacts with the relevant managing hardware. Benefits of this approach included the OS’ access to process level information that could be converted into efficient management strategies. However, the latency inherent to traversing this software stack led to power engineers working to implement hardware only power management schemes [10].

3) *Hardware-Driven Power Management*: Newer generations of hardware have fully contained power management mechanisms, which Intel describes as Hardware Managed P-states (HWP) [7]. We give an idea of the implications of hardware management through the lens of Intel, using explanations of their various power management technologies because our study is on a 14th generation Intel processor:

- **Intel Enhanced SpeedStep**: Intel SpeedStep [6] allowed for the OS to choose V and f configurations. It was introduced in 2005.
- **Intel TurboBoost**: TurboBoost [8] enables hardware to opportunistically scale up its voltage and performance to

increase computation rates when the chip is otherwise fully within its thermal limits. Intel introduced Turbo-Boost in 2008.

- **Intel SpeedShift** Intel SpeedShift [12] introduced newer hardware-only methods for power management with an OS “hint interface” for providing input to the processor, starting with the Skylake microarchitecture, which was introduced in 2015.

Given this information, we can proceed to detail our reverse engineering process.

III. UNDERSTANDING HARDWARE-MANAGED POWER

Here we conduct a case study on an Intel Core 3 100U processor embedded within an ASUS NUC 14 Pro RVKi3 Mini PC system configured to dual boot Windows 11 Pro and Ubuntu 24.04.1 LTS. The Linux kernel is version 6.8.0-48-generic. This system contains two hyperthreaded performance optimized “P-cores” and four efficiency optimized “E-cores”. We restrict the analysis in this work to a single P-core (though we could easily apply the same methodology to generalize to E-cores).

A. The Operating Point Abstraction

Key to the idea of dynamic power management in modern CPUs is the abstraction of an operating point, or a specific pairing of a voltage V and frequency f at which the processor is configured to run. This allows for a simplified digital representation of the processor’s current power dissipation state. The Advanced Configuration and Power Interface standard (ACPI) embeds the processor’s programmed operating points, and in versions 5.0 and higher of the standard use the Collaborative Processor Performance Control (CPPC) mechanism to abstract “performance levels” defined by vendor-specific mappings. To fully understand the scope of the dynamic power management mechanism for a given processor, it is useful to understand both what values are provided according to the CPPC mechanism and how these map to the given processor for that vendor. Intel implements this using an MSR named IA32_HWP_CAPABILITIES. Table I shows a snapshot of values embedded in the test processor. In the following subsections we will connect these table values explicitly to the P-core of the processor we are reverse-engineering.

Parameter	Value
Lowest Performance	1
Most Efficient Performance	25
Guaranteed Performance	16
Highest Performance	60

TABLE I: Collaborative Processor Performance Control (CPPC) configured values for the P-core of an Intel Core 3 100U Chip. Lowest Performance and Highest Performance remain static, while Most Efficient Performance changes based on the processor’s workload and Guaranteed Performance changes based on thermal and power limits.

B. MSR 0x198

A correct understanding of the processor’s operating points relies on accurate voltage and frequency readings. Without assurance of proper measurements, it is not possible to gain any understanding of how the processor makes power management decisions. In order to gain this assurance we first examined Intel’s documentation, uncovering the information that bits 47:32 of the MSR with address 0x198 were known in previous generations to the tested processor (though no longer included in the documentation for the test processor) to represent processor voltage scaled by a factor of 1/8192 [15]. This conversion produced reasonable values within the expected voltage range of the test processor but left the question of frequency measurement open.

Our initial solution to measuring the frequency, inspired by the i7z tool [17], was to use the ratio of the Intel performance events for *ref_cycles* (a counter running at a fixed frequency independent of DVFS, recovered via an open source loadable kernel module [1] from the test processor to be 2.496GHz) and *cycles* (a counter running at the frequency changing with the operating frequency of the processor), which can be shown with the following equation:

$$f = \Delta cycles / \Delta ref_cycles \quad (3)$$

Following these experiments, we realized that another 16-bit field in MSR 0x198 (bits 15:0) correlated very strongly with our computed measurements of frequency. Though referred to by terms such as “Performance State Value” in some versions of documentation it is not clear what this initially implies [16]. After further investigation we found a direct mapping between the two values related by a factor of roughly 2.56, corresponding to a frequency resolution between operating points of just under 100MHz. We display this result in Figure 1. Following this observation, we measured the cross-correlation coefficient, relating over 99% of all measured frequency values to their corresponding value in the MSR. To the best of our knowledge, the contents of this MSR have never been officially released or clarified by Intel nor proven rigorously in other research work [16].

C. Uncovering a Core’s Operating Points

Provided an accurate measurement mechanism of the processor’s voltages and frequencies, the operating points for a given core can be uncovered. Using a data collection process implemented with a tool that we will describe in the following section, we clearly observed a frequency resolution of 100 MHz between different operating points, but it was initially less clear whether minute voltage differences observed per operating point (shown in Figure 2b) were intended to be distinct operating points or an artifact of hardware management.

To resolve this question, we used the Linux *cpupower* tool to perform an experiment in which the processor’s frequency was fixed while samples were taken. This produced an output profile of voltages heavily centered around one (or two, which will be addressed later) different voltages, suggesting that these additional voltage values are not explicitly intended,

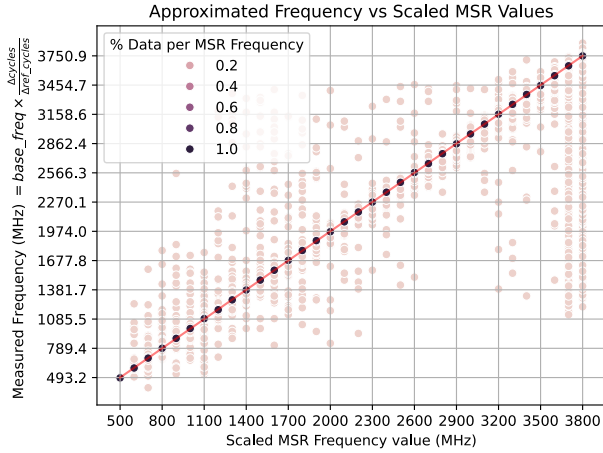


Fig. 1: Demonstrating the extremely high correlation between frequencies that we calculate from performance counter data and values in bitfield 15:0 of MSR 0x198. The Pearson correlation coefficient is 0.99969515 (p-value ≈ 0)

but rather an artifact of how the hardware control loop must toggle between frequency and voltage points. The different ranges of voltages observed for different frequencies is shown in Figure 2a. When the processor’s frequency is raised, it must first conservatively raise the voltage to ensure that no logic values will be lost. The inherent inability of the hardware PLL and voltage regulator to exactly meet software or firmware-specified discrete (V, f) points displays what is under the operating point abstraction- a large continuous space being constrained for more predictable processor performance.

D. Moving Between Operating Points

Thus far, we have demonstrated an understanding of the absolute space of operating points. But how does the processor decide to maneuver between them? Modern CPUs operate under the idea of *energy proportionality*- running the CPU at its highest capabilities when heavy computation is required and running at low frequencies (or even shutting it off) under small loads [23]. In this way, energy is saved heuristically by ensuring the CPU is not wasting energy when it does not need to perform at a high level. From data we collected, the processor primarily prefers to operate in two different regions- a lower region of frequencies from $\approx 400\text{MHz}$ to 1.1GHz and a high region of frequencies from $\approx 3.0\text{GHz}$ to 3.8GHz . Changes in operating point could be thought of as a first derivative of the absolute operating point, which we can denote as OPP' .

Via Intel’s documentation, we deduce that Intel’s dynamic power management policy is centered around adjusting the processor’s responsiveness to workloads, which, extending our notation from the previous paragraph, we describe as OPP'' , the second derivative of the operating point. If this value is low, the processor takes longer to switch between the high and low region of operating points. Presumably, OPP'' is modulated based on how much we want to weight energy conservation

versus maximal performance. This is precisely the information offered by the OS interface to the HWP mechanism via a field called the Energy Performance Preference.

1) *Energy Performance Preference*: The Energy Performance Preference (EPP) is expressed as a single byte value, where 0x0 represents a preference for performance and 0xFF represents a preference for energy conservation. By default, the value is set to the middle of the range, 0x80. We performed an experiment sweeping through the value of the EPP in order to observe the range of responsiveness for the test processor. We represent these results in Figure 3, from which it is observed that low EPP values induce an almost instant response to the processor going from low to high utilization, while high EPP values throttle the maximum operating point that can be reached and show some oscillation between significantly lower operating points for the same workload.

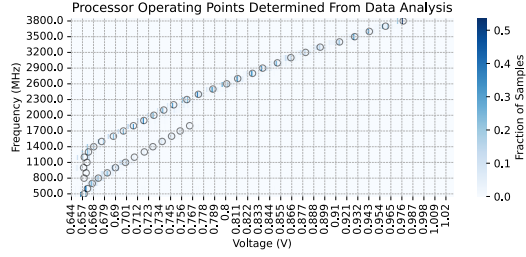
2) *Activity Window*: Another key piece of information offered by the OS to hardware interface is a field called the “activity window”. This represents a length over which the processor will dynamically account for utilization and heuristically adjust the operating point in a manner similar to the original DVFS algorithm. We omit a more detailed description of the activity window by noting that the machine learning models that we develop model the window implicitly rather than explicitly.

E. Reverse Engineering Intel’s CPPC Mapping

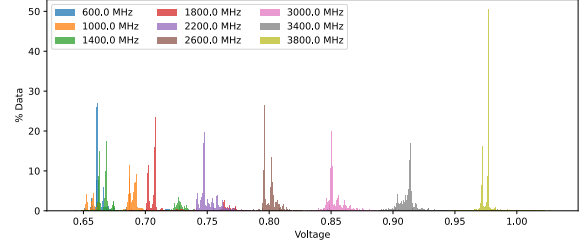
In Figure 2a we display the recovered operating points of an Intel Core 3 100U processor from our experiments. These results allow us to fully reverse engineer Intel’s CPPC implementation (mapping from an abstract performance scale to the operating points). Though not directly useful for the purpose of recreating DVFS mechanisms with proxy models, we believe this mapping is worth discussing for greater understanding of hardware managed DVFS mechanisms.

Referring to Table I, we see four different configuration values. Hints from the OS to the IA32_HWP_REQUEST register involving explicit operating points are encoded using this range. According to Intel documentation, the range [Lowest, Guaranteed] corresponds to values that are accepted through the OS to hardware interface, but can be ignored. The Guaranteed Performance value gives the frequency practically corresponding to the lowest frequency that the hardware can operate the processor at. We observe this to be 400MHz for the test processor’s P-core.

Another key observation from our data is that there is a difference between the maximum processor frequency that arises as a result of running a program and allowing the hardware to manage frequency and the maximum frequency that the processor can actually reach. All the workloads that we run on the test processor never result in an operating frequency of above 3.8GHz, but further configuration information reveals that the processor can run at up to 4.7GHz, which was confirmed in our fixed-frequency experiment detailed earlier. Because the hint register interface should need to accept any



(a) A heatmap where heat values are normalized to the fraction of the points corresponding to a frequency. Clusters are identified and labeled with circles taken to represent operating points of the Intel Core 3 100 U P-Core. Instability in voltage measurements creates the apparent multifunction of dual operating points for the lower range (see opposite panel).



(b) Here we show clusters of different voltage values measured for a fixed frequency setting. Though each frequency demonstrates a most-common value that we assume is the intended operating point, electrical characteristics of the chip affecting the dropout voltage lead to values measured in transition and due to supply voltage variation.

Fig. 2: Determining operating points using (V, f) samples

value that is *possible* for the processor to reach, we equate the Highest Performance Value (60) to this frequency.

Note that the gap between Highest Performance and Guaranteed Performance (44) corresponds very well to the gap between the processor’s lowest and highest frequencies (4.3GHz) given our measured frequency difference between operating points. Assuming a linear interpolation, we can say that for the test processor, a performance difference of 1 on the abstract CPPC scale corresponds to a frequency difference of 97.7MHz, slightly deviating from the data, from which we observed a difference of 98.72MHz (See Figure 1). This corroborates with the test processor’s measured operating points, which are heuristically documented as having a gap of 100MHz, but empirically have a gap slightly less than this. It may even be possible that this represents an off-by-one error from what was intended in vendor configuration of the processor that is carried through all the way to the operation of the processor, as the value is so close to 100MHz that this could still easily meet tolerance requirements.

IV. METHODOLOGY FOR DATA COLLECTION

After reconstructing the operating point space of our processor, we aimed to study the relationships between valued collected via performance counters for different hardware events and the corresponding processor power usage. Prior work presents power models from performance counter data [18], [21], [25]. However, much of this work relied on a limited performance counter interface provided by the hardware vendor. Models that once had to rely on measuring only one or two different processor events simultaneously can now be naturally expanded to accommodate (in our case) six performance counter events plus three additional always-on hardware performance event counters: reference cycles, cycles, and instructions. These features were added in newer generations of Intel processors’ Performance Monitoring Unit (PMU) [16].

A. Choosing Performance Counter Events

For newer processors (the test processor is an Intel Raptor Lake Refresh processor, introduced in Q1 2024 [9]) Intel offers the prospect of measuring on the order of hundreds of different events in hardware [5], and it is not immediately obvious which events should be chosen to model the hardware power management mechanism given the constraint on simultaneous measurements. To decide which events to select for our data collection mechanism, we first took the widest possible range of counter measurements using the existing perf subsystem of the Linux kernel [4], which automatically multiplexes to handle more than the hardware configured limit. This introduces the tradeoff that each individual counter value will contain more noise, but we assume that this still preserves an event’s correlation with power consumption. After greedily choosing the most correlated but distinct set of events, our event collection process chronicles how well the current workload on the processor would be performing by taking into account the performance of the memory hierarchy, front end, and rate of computation, selecting the events listed in Table II.

BR_INST_RETIRED.ALL_BRANCHES
BR_MISP_RETIRED.ALL_BRANCHES
MEM_INST_RETIRED_ANY
FRONTEND_RETIRED.ANY_DSB_MISS
DTLB_LOAD_MISSES.WALK_COMPLETED
LONGEST_LAT_CACHE.REFERENCE

TABLE II: Intel hardware performance events used in our data collection process.

B. Handling Very High Sampling Rates

Modern processors have the capability to manage power at the microsecond level, but typical data collection tools using processor performance counters and MSRs do not sample more frequently than once per millisecond due to the sampling overheads induced. This poses a massive problem in trying to accurately observe traces of operating points in a live processor, as by the Nyquist-Shannon sampling theorem, the

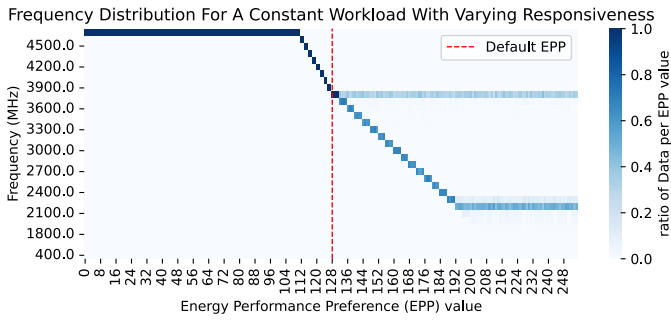


Fig. 3: Varying the hint parameter “Energy Performance Preference” via the OS to hardware hint interface offered by the test processor, demonstrated by plotting the percentage of samples are at a given frequency. Higher responsiveness implies more time at higher frequencies. Interestingly, above the midpoint, where energy management is favored, the highest possible operating point shifts from the chip maximum OPP to a lower value. Here we show a heat map where the heat values are computed by taking ratio of data per EPP.

sampling rate must be at least twice the maximum rate at which operating points change, $\max OPP$. So, in order to guarantee sampling at fine enough granularity we need a mechanism that is able to do this without inducing overheads that will disrupt the measurements themselves. In addition, the programmatic mechanisms for accessing performance counters and MSRs are not simple.

To address this challenge, we first tried a sampling mechanism from userspace using the file system abstractions that the Linux kernel provides (*i.e.*, file I/O system calls). Our userspace mechanism achieved a maximum sampling rate of 250kHz, corresponding to a 4 μ s interval between samples. Still, we faced two key challenges. First, our sampling mechanism incurred significant overhead, almost completely utilizing the processor. Second, this sampling mechanism resulted in unpredictable results with wide variations in sampling rate.

However, these experiments provided an intuition of what would make high rate sampling possible. According to documentation from Intel, this would ideally imply a sampler able to take samples on the scale of every 1-100 microseconds. It is worthwhile to note that previous applications of performance counters in performance analysis tools are not at microsecond granularity not only due to sampling overheads, but also due to a lack of necessity for such high rates for applications like software profiling, though we provide more details on this front in Section VIII.

Creating a custom Linux kernel module allowed for the possibility of working around costly and unpredictable system calls. Intel implements the RDPMC and RDMSR instructions to read performance counters and MSRs, respectively, with a latency on the order of tens of cycles. This implies that direct access to these instructions could theoretically provide a sampling rate of hundreds of MHz at the expense of eating up all CPU cycles. However, spacing these instructions out at

a granularity of 1% of total CPU cycles (*i.e.*, 1% sampling overhead) still implies a sampling rate in the high hundreds of KHz to low MHz range.

Once the fine grained sampling is enabled, a high precision timer is the other crucial component. Luckily, these are well implemented in the Linux kernel as well, specifically the family of `sleep_range()` functions, with the range itself being set to zero in order to reduce unpredictability. This family of functions does not perform timing by busy-waiting the process, unlike other functions in the Linux kernel like `*delay()`. `sleep_range()` is also more precise than the `*sleep()` family of functions at shorter intervals because the `*sleep()` family relies on the Linux kernel time abstraction of jiffies, while `sleep_range()` uses the more precise abstraction of hrtimers. This key combination of kernel level components enables our specialized and flexible data collection framework.

We document the design of our data collection mechanism in Figure 4. It spans userspace software and kernel space software, working to coordinate the simultaneous high frequency sampling with minimized overhead with efficient and latency-minimized data exfiltration from the kernel. From the userspace end, the user is able to configure which performance counters are used to collect data by piping this data to a control file provided in Linux’s `/proc` filesystem.

Importantly, we constrain our measurements to a single core in our analysis by making use of the program `taskset` to control process affinity, mapping processes precisely and deterministically to the core that we wish to operate with. Within the Linux kernel we also control kthread affinities to achieve the same effect. This ensures that the sampling kthread and the userspace process being measured *must* execute on the same core.

To make practical use of the kernel module the user can configure and trigger the sampling process to collect data from any workload using a bash script template from userspace. We avoid overhead from data exfiltration (also implemented via `/proc`) by using `taskset` to assign the exfiltrating process to unused E-cores in our Asus NUC system. This ensures that these processes do not interfere with the cores that we profiled, but it does imply the difficulty of taking measurements from many cores simultaneously.

C. Covering the Operating Point Space With Workloads

We now describe how we focused our data collection on generating datasets that would best enable the creation of proxy machine learning models for a hardware embedded DVFS mechanism. In order to run workloads that would create representative samples per operating point, we divided possible workloads into three heuristic categories that we then paired with programs that we could `taskset` to the proper core and run co-located with the sampling module:

- 1) *Compute-bound workloads*: Workloads that are bottlenecked by performing heavy computation
- 2) *I/O-bound workloads*: Workloads that are bottlenecked waiting for I/O operations including network operations, disk operations, or user input

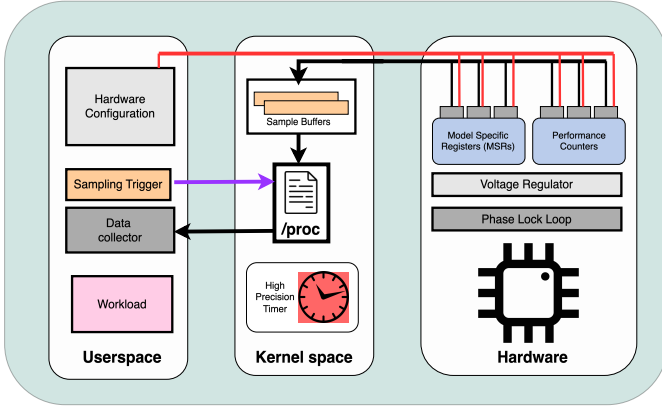


Fig. 4: Data collection mechanism used to harvest MSR and performance counter values from the hardware. Black arrows indicate the datapath from the hardware to userspace, red arrows indicate the configuration path from the userspace to the hardware, and blue arrows indicate the path between userspace and kernel space. The direction of the arrows indicates the direction that data is traveling. `/proc` is the Linux `/proc` filesystem. Configurations “pass through” the kernel, which reconfigures the proper MSRs for counting performance events.

- 3) *At-rest workloads*: Though a truly at-rest computer would enter sleep states (“C-states”) to save power, a user-focused operating system such as Ubuntu runs enough background processes to keep cores from sleeping most of the time. Through utility programs such as the `i7z` tool, it is easy to observe the percentage of time spent in each C-state, where C0 implies that the processor is fully awake and running. With only background activity we still observed that over 90% of the processor’s time would be in an awake state.

Memory-bound workloads are not included as a category because in our empirical observations on the system, workloads considered to be memory-bound did not span a significant portion of the operating points beyond the compute-heavy top operating points. This demonstrates that though memory can still pose a bottleneck to the CPU, it may not necessarily require significant speed throttling to the CPU to match its speed of operation with memory.

As representative compute bound workloads, we selected C/C++ benchmarks from the SPEC CPU 2017 benchmark suite [3] using their *ref* input, including 5 floating point “rate” versions of benchmarks {`imagick`, `nab`, `povray`, `namd`, `lbm`} and 7 integer “speed” versions of benchmarks {`deepsjeng`, `leela`, `omnetpp`, `xz`, `gcc`, `mcf`, `perl`}. As our problem space is focused on one core, we prefer “speed” benchmarks meant for single-threading but use floating point “rate” benchmarks because most of the floating point “speed” benchmarks use Fortran.

As a representative I/O bound workload, we used the Firefox browser with a prescribed schedule for using different

web applications for n minutes in order to create a wide range of data. We include the schedule in Table III.

Application	Number of Minutes
Google Search	4
Google Maps	6
YouTube	6
ChatGPT Queries	4
Image Search	4
Reading a Blog	6
Sum	30

TABLE III: Reproducible schedule for data collection from the Firefox browser. This example schedule is scaled to 30 minutes, but was at times scaled to other lengths of time as requirements/sampling parameters required.

This set of workloads was sufficient to produce at least hundreds of samples per operating point, with the more common operating points outside of the “transition region” between the high and low operating points represented by thousands, hundreds of thousands, or even millions of data points, creating an inherently biased dataset.

V. MACHINE-LEARNED POWER MODELS

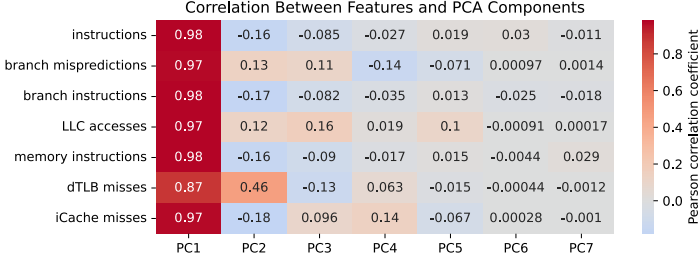
We now demonstrate our sampler module’s efficacy in supplying data to train machine learning models. Given our module we developed multiple models across different classes to evaluate how effectively we could predict HWP P-states from system statistics and windows of previous states. Our primary objective was to predict an operating point (V_{out}, f_{out}) of the system given its current state (V_{in}, f_{in}). The state of the processor was characterized using its current operating point and various performance metrics- the hardware events referenced in Section IV-A and the always-on hardware counter for retired instructions.

Though the operating point abstraction consists of a voltage and frequency pair, voltage is much noisier than frequency (see Figure 2b) and strongly correlated with frequency (correlation coefficient = 0.994). Consequently, we used frequency only to represent the operating point. This has an added advantage of reducing the size of the input and output space.

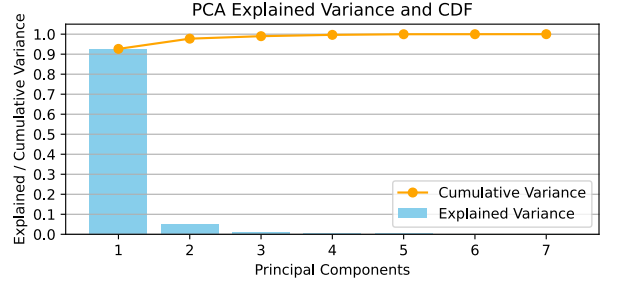
A. Data Collection

Data collection was performed in rounds with 30 samples per round, spaced approximately $50\mu s$ apart (although the exact delay provided programatically is slightly less to account for context switch time). To minimize bias introduced by the data collection process, as we expect the added intensity of sampling to eventually contribute to some increase in the operating point, we introduced cooldown periods between rounds.

The data collection method captures snapshots of the performance counters and MSRs. We computed the performance metrics by taking the delta between consecutive measurements. Register values sometimes overflow, which was handled by checking if the difference between consecutive values was negative and adding the maximum integer value to such numbers. In addition, as the data was collected



(a) PCA correlation matrix on the data we collected.



(b) PCA explained variance values.

Fig. 5: Results of PCA analysis on the data we collected with our high frequency sampling mechanism.

in rounds, we had to make sure that data processing for different rounds were kept separate. Noise was observed in the time delta within samples for a single period. We split a round into two parts if consecutive data was more than $80\mu s$ apart. For this, we needed to compute time, for which we used the following formula: $\frac{rdtsc}{base_frequency}$ (For our machine, $base_frequency = 2469MHz$). For calculating frequency and voltage, we used MSR 0x198 (see Section III-B).

The representative hardware events that we used for our data collection contain dependencies (e.g. retired instructions have a statistical correlation to retired memory instructions). To understand these dependencies, we applied principal component analysis (PCA) to the data. Figure 5 displays the result. In Figure 5a we see that the first principal component (PC1) captures $\approx 93\%$ of the variance, and adding PC2 accounts for $\approx 98\%$. Figure 5b displays that PC1 is strongly positively correlated to all features while none of the other PCs are strongly correlated to any individual feature. This means that all features themselves are correlated as well. Intuitively, this makes sense as all the features are monotonically incrementing counters attached to interconnected microarchitectural features.

B. Model Architectures

As operating points are (intended as) fixed discrete values in a continuous domain, both regression and classification are viable modeling approaches. The labels in the classification approach correspond to different operating points (i.e. 100MHz increments from 400MHz to 3.7GHz), though in some cases pruning underrepresented classes resulted in a better classifier. We used four model architectures: linear, Dense Neural Networks (DNN), Recurrent Neural Networks (RNN), and gradient-boosted decision trees.

We began with the simplest model, the linear model, as a baseline. This was shown in prior work to have reasonably strong capability [25]. To assess the impact of increasing model complexity by introducing nonlinearity we then implemented a DNN as well as an RNN, which is particularly suited for capturing temporal dependencies in time-series data. We included a non-perceptron-based gradient boosting iterative decision tree model (using the XGBoost library) for additional comparison. While we considered even more

sophisticated architectures such as GRU and LSTM RNNs and Transformers, we found that our models could already fit the data accurately with less storage and complexity than these models.

The input features consisted of 8 variables: 7 performance counters and 1 previous frequency value. Given the small interval between samples, a single data point would be insufficient for accurately predicting the next operating point. Therefore, we utilized 10 consecutive samples as input for training our machine learning models. For non-RNN models, this results in an input dimension of $8 \times 10 = 80$. While RNNs can handle sequences of varying lengths, we used a fixed sequence length of 10 for training and testing. To speed up training, the RNN models were configured in a sequence-to-sequence format, producing an output for each time step. However, during evaluation, only the final output was considered, ensuring comparability with the other models. The regression models have a 1 neuron linear output, whereas classifiers have 34 neuron softmax outputs (representing frequency classes in 100MHz increments as mentioned previously). A comprehensive documentation of the models we used, including total model size is contained in Table IV.

C. Model Training and Results

Data was partitioned into 90% for training and 10% for testing. We ensure that all operating points are present in the test data. The regression and classification models minimize *mean squared error* and *categorical cross-entropy* respectively. To address bias in the dataset, which was skewed toward certain operating points (e.g., 1.1GHz for idle and 3.8GHz for the default maximum), we reweighted the loss functions to ensure equal representation of all operating points during training.

We trained the models for a maximum of 400 epochs (though some converged within 100 epochs). Performance for regression models was evaluated using the R^2 score, while classification models were assessed using the macro F1 score. Here, macro F1 score is the multi-class version of F1 score, which is the unweighted mean of the F1 scores for each label (while the F1 score itself is the harmonic mean of classifier precision and recall). Results for all models are encapsulated in Table V.

Model	Input Dimension	Hidden Layers	Regression/Classifier	Total Model Size (bytes)
Linear	80	no hidden layer	R	984
			C	33,056
DNN	80	Layer 1: Dense, 50 neurons, ReLU activation Layer 2: Dense, 50 neurons, ReLU activation	R	26,604
			C	100,016
RNN	8	Layer 1: Simple RNN, 5 neurons, tanh activation Layer 2: Dense, 10 neurons, ReLU activation	R	564
			C	6,060
Gradient boosting	80	100 decision trees, each with max depth 5	R	257,732
			C	5,240,249

TABLE IV: Model architectures and parameter sizes

Model	R^2 Score (Regression)	Macro F1 Score (Classification)
Linear	0.9989	0.2593
DNN	0.9988	0.8853
RNN	0.9986	0.9198
Gradient boosting	0.9993	0.9182

TABLE V: Evaluation metrics for different machine learning model classes. Classifiers are evaluated with the Macro F1 Score while regressors are evaluated with the R^2 coefficient.

Model	R/C	Avg. Per-OP Dist	Max. Per-OP Dist
Linear	R	0.5670	1.2917
DNN	R	0.593513	1.125
RNN	R	0.4669	1.2162
Gradient boosting	R	0.6208	1.2432
Linear	C	1.3472	3.50
DNN	C	0.2201	0.9583
RNN	C	0.1809	0.75
Gradient boosting	C	0.1830	0.75

TABLE VI: Measuring the maximum and average per-class errors for the models in Table V

a) *Regression vs. Classification*: To compare regression and classification models, we rounded regression outputs to the nearest discrete operating point and calculated the distance between predicted and true operating points. To address dataset bias, we measured the average distance per operating point and computed the overall average and maximum of these distances. Table VI contains these results. Here we can see that classifiers give both smaller average and maximum distances from the correct operating point than regression models. However, the linear model performs very poorly in a classification setup, but in regression remains competitive with other models. This demonstrates that R^2 does not compare directly with the F1 score metric. Though all regression models have very strong fit they are unable to reach the granularity of the classification models, which are able to obtain a *maximum* per-operating point distance of less than a single operating point.

VI. MODEL STEALING ATTACKS

The highly accurate models for dynamic power consumption that we constructed comprise a model stealing attack that highlights the dangers of relying on a confidential machine learning model to manage power. The resolution of what can be viewed within in the processor is increasing, both in terms of more physical performance counters (often per-core) as well as high frequency and low overhead sampling pro-

vided by necessary mechanisms for OS system management. This creates a significant mismatch with intellectual property management strategies by vendors, making the problem of embedding a completely confidential machine learning model for power management in hardware untenable. Mechanisms for concealment in hardware such as logic locking will not be sufficient, as we expect that reverse engineering the architecture of the machine learning model is made simple by the constrained space for power management itself. It would be difficult to disguise performance counters and/or power feedback mechanisms without simply revealing their values as well.

VII. APPLICATION TO CRYPTOGRAPHIC POWER ATTACKS

We now describe an additional application of our work to cryptographic power side channel attacks. Cryptographic keys are considered the gold standard of confidential information because security breaches that can obtain them enable unfettered access to all information protected by the relevant key(s) [24]. In the past, power sampling to obtain cryptographic keys required direct hardware access [20], but an ultrafast sampling module such as the one used in this work has the potential to elevate cryptographic key stealing to the software level. This lowers the barrier of entry to perform such attacks.

Beyond the operating point abstraction, dynamic processor workloads can insert noise into the power management control loop, which we term “sub-operating point variance.” This sub-operating point variance is not random, but correlates with data-dependent properties such as Hamming weight (i.e. the number of ones in a binary representation of the data), opening a new avenue for the study of security in power management. This was recently demonstrated in the HertzBleed attack [33], in which sub-operating point variance in post-quantum cryptographic workloads was used to discover cryptographic keys. Crucially, though the work frames its results in terms of changing operating point distributions, the observable power differences come at the sub-operating point level, which has not been explored in prior work.

Using the high resolution of our previously investigated MSR, we conducted high frequency sampling in order to reveal sub-operating point variances in cryptographic operations using the well known asymmetric cryptography scheme RSA [27]. These variances are expected due to the differing Hamming weights in the private key that will induce power

variations as a cryptographic process is carried out [33]. In Figure 6, we demonstrate sub-operating point variances in RSA signature verification operations using keys with Hamming weights that differ by $\approx 100b$ (minimum: 972, maximum: 1083). A clear pattern can be seen in the power consumption, which fluctuates within $\approx 5Hz$, much less than the distance between operating points. This is a useful proof-of-concept of a powerful attack with our sampler module.

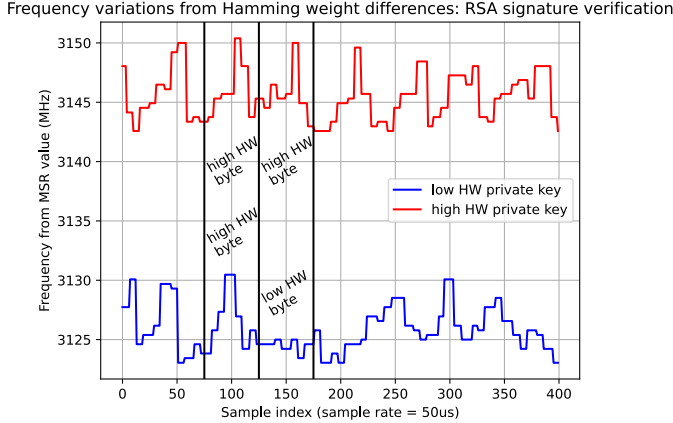


Fig. 6: Sampling an RSA signature verification with our high frequency sampling module. Regular peaks occur for approximately each byte of the private key, which is 2048b. 100 RSA keypairs were generated, and the private key with the lowest and highest Hamming weight were selected for testing. Note that the high Hamming weight key trace is offset for visibility.

VIII. RELATED WORK

A. Tools For Measuring System Statistics

On Linux, `perf` [4] is the default performance counter measurement tool. However, despite its massive configurability, the tool explicitly recommends not sampling more frequently than 1ms due to sampling overheads because it is not designed for higher frequency sampling- likely due to a heavy use of system calls to manage the counters. [2] profiles the overheads of different `perf` configurations, demonstrating the heavy overheads of multiplexing and high performance variability over the space of the `perf` tool. `i7z` [17] is a useful interactive monitoring tool that includes many power statistics, but only samples once per second, and only estimates frequency, whereas in our work we are able to reverse engineer MSRs for determining frequencies directly. `LIKWID` [32] is another performance counter measurement tool, meant to be lightweight, but relies on reads and writes to the `/dev/msr` [19] interface, which will also incur overheads that are unnecessary for high frequency sampling. None of these tools are geared toward a high frequency application, typically relying on summary statistics. In addition, our tool does not attempt to multiplex between sets of performance counters, a technique that limits the sampling rate by increasing measurement overhead and

decreasing the resolution at which system statistic changes can be observed.

B. Power-Based Attacks

Power has been used as a vector for security attacks in many prior works. In side-channel attacks, power usage correlates to data use, leaking data values when the computer performs security-sensitive operations. With a power virus [13], methods are developed to make a processor run at its absolute maximum power level in order to damage the processor. On ARM processors, prior work [30] uses an unrestricted software interface to the power management hardware to induce faults that can break hardware. Additionally, many frameworks exist for stress-testing and workload cloning, using mechanisms such as gradient descent and genetic algorithms [14],[26]. However, these frameworks tend to focus on simple objectives such as maximized power consumption or dI/dt . In contrast, this work examines the space of hardware attacks enabled by a combination of customized processor understanding, enhanced data collection ability, and machine learning models built from this data in order to capture the essence of the mechanisms' functionality.

C. Power Models

Performance counters have also been used to construct power models in much prior work. Joseph and Martonosi [18] built power models to heuristically attribute counter values to per-functional unit power consumption. Rajamani et al. [25] created linear regression power models with much more constrained numbers of counters in hardware. Lim et al. [21] built a power estimation framework using a modified regression framework. In these works hardware capabilities had simply not scaled to the point at which we conducted our analysis, in which a rich variety of models built from customized sets of performance counters can reap the greatest gains. Other works construct power models using deep learning [22] or reinforcement learning [31] for different platforms such as GPUs.

IX. CONCLUSION

Dynamic power management is a key component of modern processors. Using increased visibility into closed-source hardware (via reverse engineering an Intel Core 3 100 U Processor) we design a high frequency customized sampling module for recording traces of operating points. This mechanism in turn enables the stealing of power management machine learning models and exposes an inherent incompatibility between data visibility mechanisms and confidential models in hardware. In the future, techniques for detecting these attacks (such as watermarking or anomaly detection) and preventing them (such as dynamically adjusting the utility of the power model itself) require exploration. Solutions across the stack will be necessary to balance between visibility to users and protection of IP from model stealing attacks.

X. ACKNOWLEDGEMENTS

Simha Sethumadhavan has a significant financial interest in Chip Scan, Inc.

REFERENCES

- [1] Trail of Bits. *A tsc_freq_khz driver for everyone*. 2019. URL: https://github.com/trailofbits/tsc_freq_khz.
- [2] Georgios Bitzes and Andrzej Nowak. *The overhead of profiling using PMU hardware counters*. CERN Openlab Report. CERN, 2014. URL: <https://download.intel.com/design/network/papers/30117401.pdf>.
- [3] James Bucek, Klaus-Dieter Lange, and Jóakim v. Kistowski. “SPEC CPU2017: Next-generation compute benchmark”. In: *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*. 2018, pp. 41–42.
- [4] Linux perf wiki contributors. *perf: Linux profiling with performance counters*. Dec. 2024.
- [5] Intel Corporation. 2024. URL: <https://perfmon-events.intel.com/>.
- [6] Intel Corporation. *Enhanced Intel® SpeedStep® Technology for the Intel® Pentium® M Processor*. White Paper. Intel Corporation, Mar. 2004. URL: <https://download.intel.com/design/network/papers/30117401.pdf>.
- [7] Intel Corporation. *Intel 64 and IA-32 Architecture Software Developer’s Manual: Combined Volumes 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, 4*. 2010.
- [8] Intel Corporation. *Intel turbo boost technology in Intel core microarchitecture (Nehalem) based processors*. White Paper. Nov. 2008.
- [9] Intel Corporation. *Intel® Core™ 3 processor 100U*. 2024. URL: <https://www.intel.com/content/www/us/en/products/sku/236776/intel-core-3-processor-100u-10m-cache-up-to-4-70-ghz/specifications.html>.
- [10] Intel Corporation. *Power Management - Technology Overview*. Technology Guide. Intel Corporation, Feb. 2022. URL: <https://networkbuilders.intel.com/docs/networkbuilders/power-management-technology-overview-technology-guide-1677587938.pdf>.
- [11] Linux kernel official documentation. *CPU Performance Scaling*. 2017. URL: <https://docs.kernel.org/admin-guide/pm/cpufreq.html>.
- [12] Jack Doweck et al. “Inside 6th-Generation Intel Core: New Microarchitecture Code-Named Skylake”. In: *IEEE Micro* 37.2 (2017), pp. 52–62. DOI: 10.1109/MM.2017.38.
- [13] Karthik Ganesan and Lizy K. John. “MAximum Mul-ticore POwer (MAMPO): an automatic multithreaded synthetic power virus generation framework for multi-core systems”. en. In: *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. Seattle Washington: ACM, Nov. 2011, pp. 1–12. ISBN: 978-1-4503-0771-0. DOI: 10.1145/2063384.2063455. URL: <https://dl.acm.org/doi/10.1145/2063384.2063455> (visited on 09/22/2024).
- [14] Zacharias Hadjilambrou et al. “GeST: An Automatic Framework For Generating CPU Stress-Tests”. In: *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Mar. 2019, pp. 1–10. DOI: 10.1109/ISPASS.2019.00009. URL: <https://ieeexplore.ieee.org/document/8695639/?arnumber=8695639> (visited on 09/24/2024).
- [15] Intel. *Intel Software Developer Community*. 2019. URL: <https://community.intel.com/t5/Software-Tuning-Performance/How-to-determine-cause-of-processor-frequency-scale-down-to-200m-p/1137067>.
- [16] “Intel® 64 and IA-32 Architectures Software Developer’s Manual, Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4”. en. In: (2024). URL: <https://cdrdv2.intel.com/v1/dl/getContent/671200>.
- [17] Abhishek Jaientilal. *i7z tool*. 2010. URL: <https://code.google.com/archive/p/i7z/>.
- [18] Russ Joseph and Margaret Martonosi. “Run-time power estimation in high performance microprocessors”. en. In: *Proceedings of the 2001 international symposium on Low power electronics and design - ISLPED ’01*. Huntington Beach, California, United States: ACM Press, 2001, pp. 135–140. ISBN: 978-1-58113-371-4. DOI: 10.1145/383082.383119. URL: <http://portal.acm.org/citation.cfm?doid=383082.383119> (visited on 11/23/2024).
- [19] Michael Kerrisk. *msr(4) – Linux manual page*. Linux/UNIX system programming training. May 2024.
- [20] Paul Kocher, Joshua Jaffe, and Benjamin Jun. “Differential power analysis”. In: *Advances in Cryptology—CRYPTO’99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings* 19. Springer. 1999, pp. 388–397.
- [21] Min Yeol Lim, Allan Porterfield, and Robert Fowler. “SoftPower: fine-grain power estimations using performance counters”. en. In: *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*. Chicago Illinois: ACM, June 2010, pp. 308–311. ISBN: 978-1-60558-942-8. DOI: 10.1145/1851476.1851517. URL: <https://dl.acm.org/doi/10.1145/1851476.1851517> (visited on 09/30/2024).
- [22] Nadjib Mammeri et al. “Performance Counters based Power Modeling of Mobile GPUs using Deep Learning”. In: *2019 International Conference on High Performance Computing & Simulation (HPCS)*. July 2019, pp. 193–200. DOI: 10.1109/HPCS48598.2019.9188139. URL: <https://ieeexplore.ieee.org/document/9188139/> (visited on 09/29/2024).
- [23] David Meisner, Brian T Gold, and Thomas F Wenisch. “The powernap server architecture”. In: *ACM Transactions on Computer Systems (TOCS)* 29.1 (2011), pp. 1–24.
- [24] Lily Hay Newman. *The Comedy of Errors That Let China-Backed Hackers Steal Microsoft’s Signing Key*. URL: <https://www.intel.com/content/www/us/en/>

products/sku/236776/intel-core-3-processor-100u-10m-cache-up-to-4-70-ghz/specifications.html.

- [25] Karthik Rajamani et al. *Online Power and Performance Estimation for Dynamic Power Management*. IBM Research Report. IBM Corporation, July 2006. URL: <https://download.intel.com/design/network/papers/30117401.pdf>.
- [26] Gokul Subramanian Ravi et al. “MicroGrad: A Centralized Framework for Workload Cloning and Stress Testing”. In: *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Mar. 2021, pp. 70–72. DOI: 10.1109/ISPASS51385.2021.00018. URL: <https://ieeexplore.ieee.org/document/9408182/?arnumber=9408182> (visited on 09/24/2024).
- [27] Ronald L Rivest, Adi Shamir, and Leonard Adleman. “A method for obtaining digital signatures and public-key cryptosystems”. In: *Communications of the ACM* 21.2 (1978), pp. 120–126.
- [28] Efraim Rotem. *Intel® Energy Efficiency Architecture*. Talk presented at Hot Chips Symposium. Presentation slides available at https://hc2023.hotchips.org/assets/program/conference/day1/Platforms/Energy_Efficiency_Hotchips2023_Rev_2_Final.pdf, Presentation video: <https://youtu.be/g3-4K1UCpoU&t=3600s>. 2023. URL: <https://hc2023.hotchips.org/#conference-day1>.
- [29] Etienne Le Sueur and Gernot Heiser. “Dynamic Voltage and Frequency Scaling: The Laws of Diminishing Returns”. en. In: *Proceedings of the 2010 international conference on Power aware computing and systems*. 2010.
- [30] Adrian Tang, Simha Sethumadhavan, and Salvatore Stolfo. “CLKSCREW: Exposing the Perils of Security-Oblivious Energy Management”. en. In: *26th USENIX Security Symposium*. 2017.
- [31] Zhongyuan Tian et al. “Multi-Core Power Management through Deep Reinforcement Learning”. In: *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. ISSN: 2158-1525. May 2021, pp. 1–5. DOI: 10.1109/ISCAS51556.2021.9401447. URL: <https://ieeexplore.ieee.org/document/9401447/?arnumber=9401447> (visited on 10/07/2024).
- [32] Jan Treibig, Georg Hager, and Gerhard Wellein. “LIK-WID: Lightweight Performance Tools”. en. In: *2010 39th International Conference on Parallel Processing Workshops*. arXiv:1104.4874 [cs]. Sept. 2010, pp. 207–216. DOI: 10.1109/ICPPW.2010.38. URL: <http://arxiv.org/abs/1104.4874> (visited on 10/20/2024).
- [33] Yingchen Wang et al. “Hertzbleed: Turning power {Side-Channel} attacks into remote timing attacks on x86”. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 679–697.
- [34] Mark Weiser et al. “Scheduling for Reduced CPU Energy”. en. In: *USENIX 1994 Operating Systems Design and Implementation Proceedings*. Ed. by Tomasz Imielinski and Henry F. Korth. Series Title: The Kluwer International Series in Engineering and Computer Science. Springer US, 1994, pp. 449–471. DOI: 10.1007/978-0-585-29603-6_17. (Visited on 10/01/2024).
- [35] Jawad Haj Yahya et al. *AgileWatts: An Energy-Efficient CPU Core Idle-State Architecture for Latency-Sensitive Server Applications*. en. arXiv:2203.02550 [cs]. Oct. 2022. URL: <http://arxiv.org/abs/2203.02550> (visited on 10/01/2024).